New Relic APM — Day 2 Lab Guide

Oct 28, 2026 · @Vivek Arora

Hands-on labs for *Error analytics, alerting and incident response* · easyTravel on Linux · New Relic Java agent

How to use this guide

Each lab matches a module of the New Relic APM — Day 2 Study Guide: read the module, then do its lab. The labs build on the Day 1 lab system — easyTravel in Docker with the New Relic Java agent in the frontend and backend Tomcats.

Lab	Title	Module	Time
2.1	Triage a burst of exceptions and identify the introducing release	Module 4	60 min
2.2	Convert a noisy static alert into a tuned baseline NRQL condition	Module 5	60 min
2.3	Run a simulated P1 incident from alert to acknowledgement to root cause	Module 6	60 min

Before you start

- Day 1 Lab 1.0 is complete: `etdc` (the Compose alias) and `pattern on|off <Name>` work in your shell, and both services report to New Relic.
- The Day 1 reset is done: all problem patterns off, MongoDB running, distributed tracing on in both `newrelic.yml` files.
- The New Relic CLI profile from Lab 1.3 exists (`newrelic profile list`).
- Your New Relic user can create **workloads, alert policies, conditions, destinations, workflows and muting rules**, and you can receive email at the address you will use as a destination.

```
cd ~/nr-lab
etdc ps --format "table {{.Name}}\t{{.Status}}"
for p in DBSpamming DBSpammingAuthWithAppDeployment JourneySearchError500
TravellersOptionBox; do pattern off $p; done
```

Conventions

- `$` lines run on the lab VM as `trainee`, from `~/nr-lab` unless a step says otherwise.
- **Expected output** pictures show terminal output. **Expected view** pictures are redrawn New Relic screens: names, numbers and error classes on your screen will differ — look for the shape (a new group, a step after a marker, a band, one issue with three incidents).
- Restarting the frontend or backend container **resets easyTravel's problem patterns**. Whenever a step restarts a container, it switches the patterns it needs back on afterwards.
- Alerting needs time: conditions evaluate one window at a time and wait for late data. When a step says *wait*, use the time to read the matching study-guide section.

Lab 2.1 — Triage a burst of exceptions and identify the introducing release

Goal: release 2.2.0 of easyTravel ships a bug. Use Errors Inbox to find the new error group, read its stack trace and attributes, prove with NRQL which release introduced it, take ownership, mark a known error as expected, and close the group with release 2.2.1.

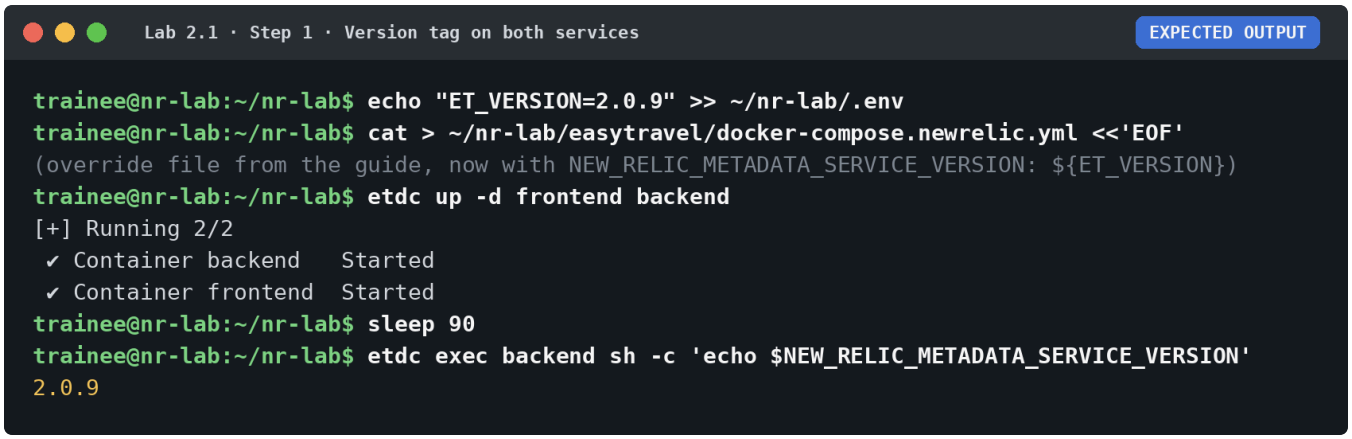
Time: 60 minutes. **Module:** 4.

Step 1 — Tag both services with a version

Errors Inbox reads the release from `tags.service.version`. The Java agent sets it from the `NEW_RELIC_METADATA_SERVICE_VERSION` environment variable. Rewrite the Compose override from Day 1 with that variable added (everything else is unchanged):

```
echo "ET_VERSION=2.0.9" >> ~/nr-lab/.env
cat > ~/nr-lab/easytravel/docker-compose.newrelic.yml <<'EOF'
services:
  backend:
    environment:
      CATALINA_OPTS: "-javaagent:/opt/newrelic/newrelic.jar -
Dconfig.apmServerDefault=APM -Xmx300m"
      NEW_RELIC_LICENSE_KEY: ${NEW_RELIC_LICENSE_KEY}
      NEW_RELIC_METADATA_SERVICE_VERSION: ${ET_VERSION}
    volumes:
      - ../agent/backend:/opt/newrelic
  frontend:
```

```
    environment:
      CATALINA_OPTS: "-javaagent:/opt/newrelic/newrelic.jar -
Dconfig.apmServerDefault=APM -Xmx525m"
      NEW_RELIC_LICENSE_KEY: ${NEW_RELIC_LICENSE_KEY}
      NEW_RELIC_METADATA_SERVICE_VERSION: ${ET_VERSION}
    volumes:
      - ../agent/frontend:/opt/newrelic
EOF
etdc up -d frontend backend
sleep 90
etdc exec backend sh -c 'echo $NEW_RELIC_METADATA_SERVICE_VERSION'
```



Release 2.0.9 is the rollback you recorded at the end of Lab 1.3, so the version history in New Relic is consistent.

Step 2 — Create a workload and open Errors Inbox

1. In New Relic, open **Workloads → Create a workload**. Name it `easyTravel`, add **easyTravel-frontend** and **easyTravel-backend**, and save.
2. Open **Errors Inbox** and choose the **easyTravel** workload in the scope selector at the top.
3. Keep the default filter (`error.expected != true`) and set the time picker to *Last 3 hours*.

Step 3 — A known, noisy error

The search team has a known problem: some journey searches fail with HTTP 500. It has been in production for weeks and is tracked in their backlog.

```
pattern on JourneySearchError500
date +%H:%M
```

Wait 5–10 minutes. A group for this error appears in Errors Inbox with steady occurrences. Open it, note its **error class** and a phrase from its **message** exactly (you need them in Step 7), and close the panel.

Step 4 — Ship release 2.2.0

Release 2.2.0 changes the traveller options box. Deploy it, record it, and — because a restart resets the patterns — switch the known error back on together with the new bug:

```
sed -i 's/^ET_VERSION=.*/ET_VERSION=2.2.0/' ~/nr-lab/.env
etdc up -d frontend backend && sleep 90
for app in easyTravel-frontend easyTravel-backend; do
  newrelic changeTracking create \
    --entitySearch "name = '$app' AND domain = 'APM' AND type =
'APPLICATION'" \
    --category Deployment --type Basic --version "2.2.0" \
    --description "Release 2.2.0: traveller options box" --user "$USER" >
/dev/null && echo "marker: $app 2.2.0"
done
pattern on JourneySearchError500
pattern on TravellersOptionBox          # the bug shipped in 2.2.0
date +%H:%M
```

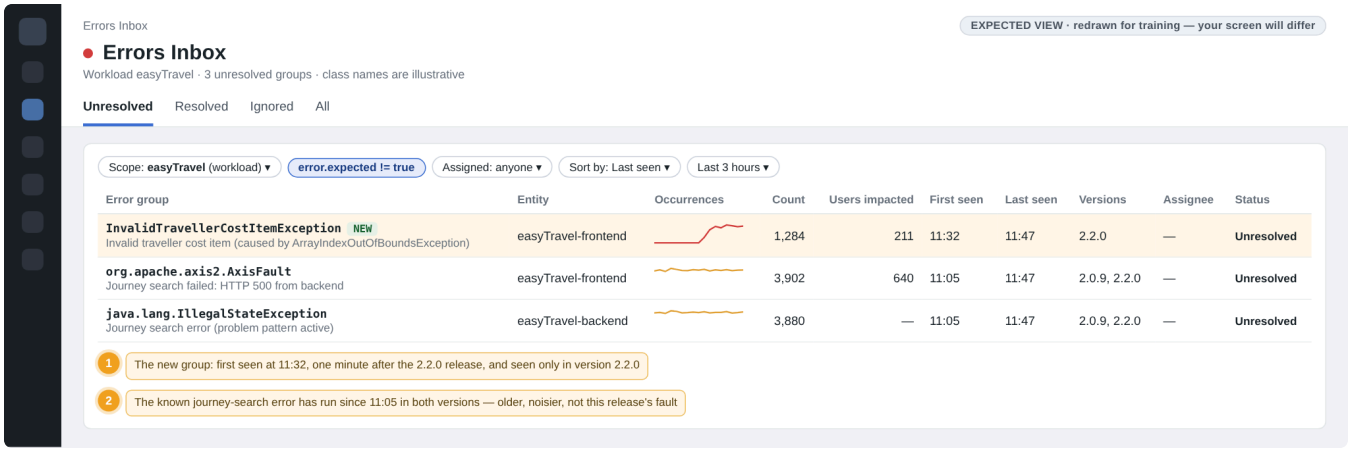
Lab 2.1 · Step 4 · Ship release 2.2.0 — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ sed -i 's/^ET_VERSION=.*/ET_VERSION=2.2.0/' ~/nr-lab/.env
trainee@nr-lab:~/nr-lab$ etdc up -d frontend backend && sleep 90
[+] Running 2/2
 ✔ Container backend   Started
 ✔ Container frontend  Started
trainee@nr-lab:~/nr-lab$ for app in easyTravel-frontend easyTravel-backend; do newrelic
                         changeTracking create --entitySearch "name = '$app' AND domain =
                         'APM' AND type = 'APPLICATION'" --category Deployment --type Basic
                         --version "2.2.0" --description "Release 2.2.0: traveller options
                         box" --user "$USER" > /dev/null && echo "marker: $app 2.2.0"; done
marker: easyTravel-frontend 2.2.0
marker: easyTravel-backend 2.2.0
trainee@nr-lab:~/nr-lab$ pattern on JourneySearchError500
JourneySearchError500 enabled=true -> HTTP 200
trainee@nr-lab:~/nr-lab$ pattern on TravellersOptionBox
TravellersOptionBox enabled=true -> HTTP 200
trainee@nr-lab:~/nr-lab$ date +%H:%M
11:31
```

Wait 10 minutes. The load generator books journeys continuously, so the new exception arrives as a burst.

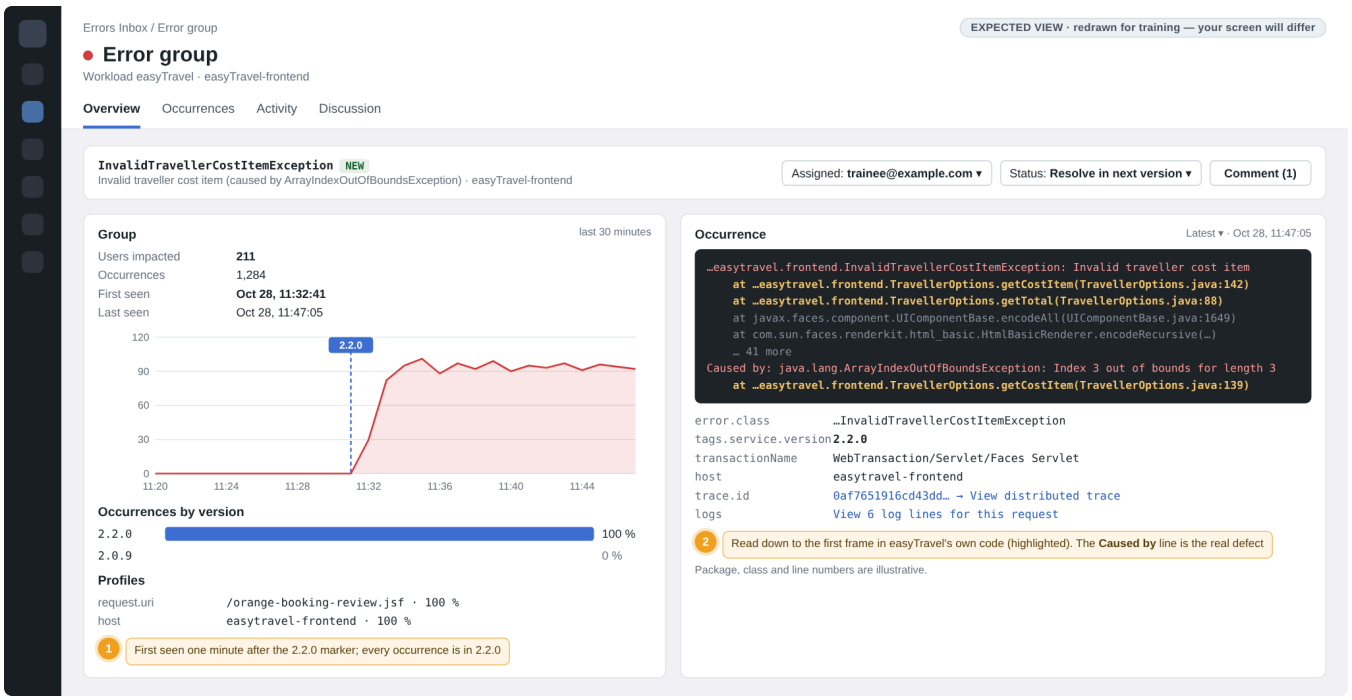
Step 5 — Triage the new group

In Errors Inbox (scope *easyTravel*, last hour), a **new** group sits at the top, with a high occurrence count and *first seen* a few minutes after your release.



Open it and work through the panel:

1. **Left (the group):** *users impacted*, *first seen* / *last seen*, the occurrences chart (the 2.2.0 marker sits just before the first occurrence), **Occurrences by version** (only 2.2.0), and the most common attribute values (which transaction, which host).
2. **Right (one occurrence):** the **stack trace** — read it from the top until you reach the first frame in easyTravel's own code; that frame is where to fix it. Note the *Caused by* line: the real cause is an `ArrayIndexOutOfBoundsException` wrapped in the application's own exception.
3. Open the **distributed trace** and **logs** of the occurrence to see the request that failed.



Now take ownership:

- **Assign** the group to yourself.
- **Comment:** `Introduced in 2.2.0 (traveller options box). Index error in cost items. Fix planned in 2.2.1.`
- Set the status to **Resolve in next version**.

Question 1. The group shows 1,200 occurrences but 210 users impacted. Which number decides the priority, and why?

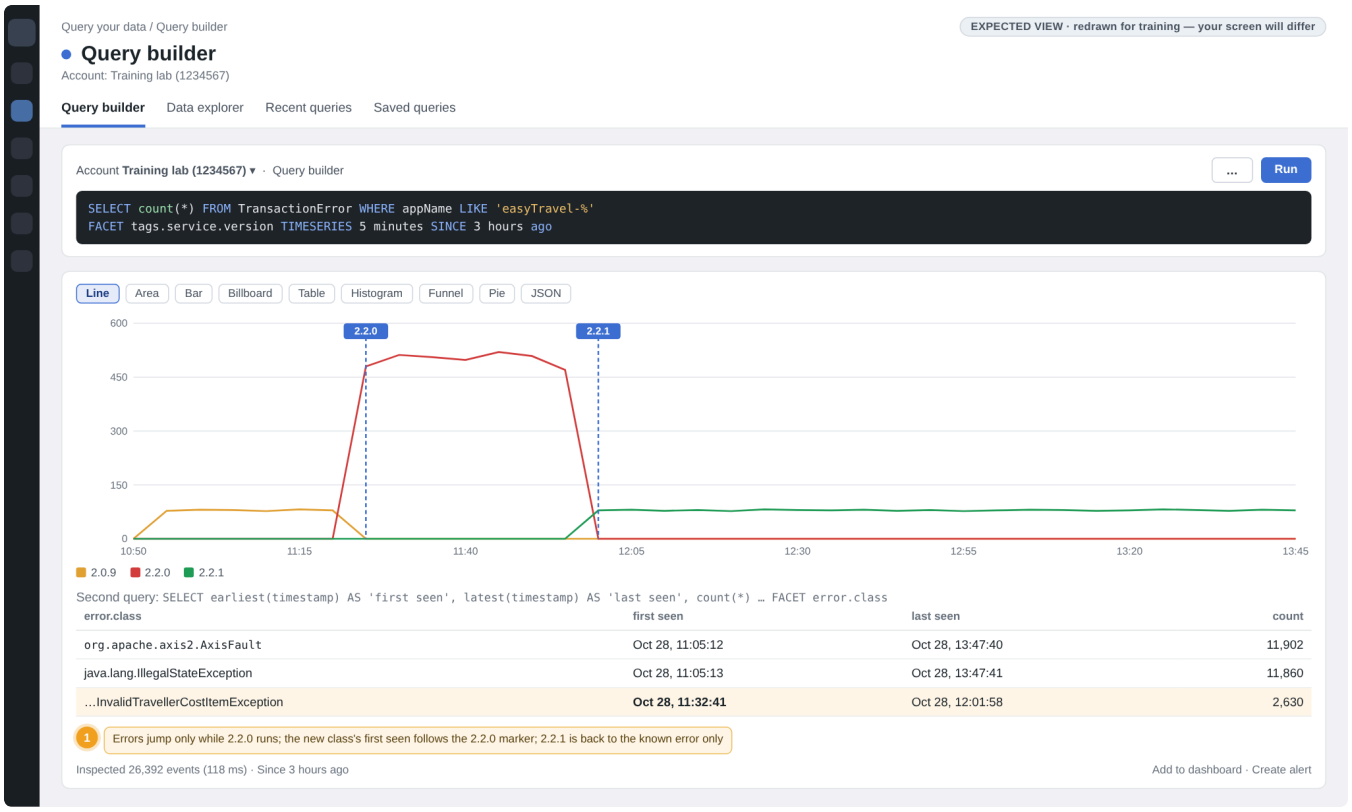
Step 6 — Prove the introducing release with NRQL

```
-- Errors by version and class
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%'
FACET tags.service.version, error.class SINCE 3 hours ago

-- First and last occurrence of each error class
SELECT earliest(timestamp) AS 'first seen', latest(timestamp) AS 'last seen',
count(*)
FROM TransactionError WHERE appName LIKE 'easyTravel-%' FACET error.class
SINCE 3 hours ago

-- Errors over time by version, with the deployment markers on the chart
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%'
FACET tags.service.version TIMESERIES 5 minutes SINCE 3 hours ago

-- Which transaction and which host
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%' AND
error.class LIKE '%TravellerCost%'
FACET transactionName, host SINCE 1 hour ago
```



The three facts agree: *first seen* follows the 2.2.0 marker, the error exists only in 2.2.0, and 2.0.9 had the same traffic without it. Release 2.2.0 introduced it.

> If `tags.service.version` is empty, check Step 1 (`etdc exec … echo $NEW_RELIC_METADATA_SERVICE_VERSION`) and that the containers were recreated with `up -d`, not just restarted.

Step 7 — Mark the known error as expected

The journey-search 500s are known and owned elsewhere. They inflate the error rate and would trigger the alerts you build in Labs 2.2 and 2.3. Mark them as **expected** in both agents. The setting is dynamic: no restart is needed, so no patterns are reset.

Look at the group's error class first:

- If the class is **specific** to this problem (an application exception such as `…JourneySearchException`), use `expected_classes`.
- If the class is **generic** — `org.apache.axis2.AxisFault`, `RuntimeException`, or a bare status code — many different failures share it, including the outage you will cause in Lab 2.3. Marking the class expected would hide them all. Use `expected_messages` instead: the class **plus** a phrase that only this error's message contains.

The lab uses the safer message form:

```
CLASS='<error class from Step 3>'
MSG='<a phrase that only this error message contains>'
for t in frontend backend; do
  sed -i "/^  log_level: info/a\  error_collector:\n    expected_messages:\n
\"$CLASS\":\n        - \"$MSG\"" ~/nr-lab/agent/$t/newrelic.yml
done
grep -A3 "error_collector:" ~/nr-lab/agent/frontend/newrelic.yml
```

Lab 2.1 · Step 7 · Mark the known error as expected — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ CLASS='org.apache.axis2.AxisFault'
trainee@nr-lab:~/nr-lab$ MSG='Journey search failed'
trainee@nr-lab:~/nr-lab$ for t in frontend backend; do sed -i "/^  log_level: info/a\
                           error_collector:\n    expected_messages:\n      \"$CLASS\":\n
                                   - \"$MSG\"" ~/nr-lab/agent/$t/newrelic.yml; done
trainee@nr-lab:~/nr-lab$ grep -A3 "error_collector:" ~/nr-lab/agent/frontend/newrelic.yml
  error_collector:
    expected_messages:
      "org.apache.axis2.AxisFault":
        - "Journey search failed"
dynamic setting: the running agents pick it up, no restart and no pattern reset
```

> If the group's class is only a status code (for example `500`) and the message is not specific, do **not** add `expected_status_codes: 500` — that would hide every 500, including new bugs. Leave it unexpected and set the group to **Ignored** in Errors Inbox instead.

Wait 5 minutes, then check:

```
SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%'
FACET error.expected, error.class SINCE 15 minutes ago
```

New occurrences of the known error arrive with `error.expected = true`; the APM error rate no longer counts them; and with the default filter, Errors Inbox hides the group once all its recent occurrences are expected (a mixed group is tagged *contains expected*).

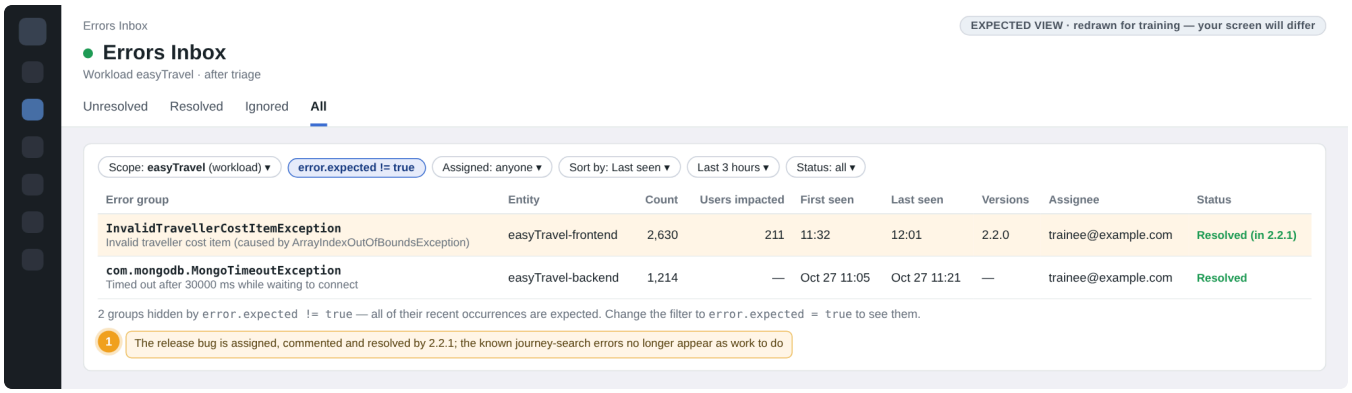
Step 8 — Ship the fix in 2.2.1

```
pattern off TravellersOptionBox
sed -i 's/^ET_VERSION=.*/ET_VERSION=2.2.1/' ~/nr-lab/.env
etdc up -d frontend backend && sleep 90
for app in easyTravel-frontend easyTravel-backend; do
  newrelic changeTracking create \
    --entitySearch "name = '$app' AND domain = 'APM' AND type =
'APPLICATION'" \
    --category Deployment --type Basic --version "2.2.1" \
    --description "Release 2.2.1: fix traveller cost items" --user "$USER" >
/dev/null && echo "marker: $app 2.2.1"
done
pattern on JourneySearchError500        # still running in production, now
expected
```

Lab 2.1 · Step 8 · Ship the fix in 2.2.1 — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern off TravellersOptionBox
TravellersOptionBox enabled=false -> HTTP 200
trainee@nr-lab:~/nr-lab$ sed -i 's/^ET_VERSION=.*/ET_VERSION=2.2.1/' ~/nr-lab/.env
trainee@nr-lab:~/nr-lab$ etdc up -d frontend backend && sleep 90
[+] Running 2/2
 ✔ Container backend   Started
 ✔ Container frontend  Started
trainee@nr-lab:~/nr-lab$ for app in easyTravel-frontend easyTravel-backend; do newrelic
                           changeTracking create … --version "2.2.1" …; done
marker: easyTravel-frontend 2.2.1
marker: easyTravel-backend 2.2.1
trainee@nr-lab:~/nr-lab$ pattern on JourneySearchError500
JourneySearchError500 enabled=true -> HTTP 200
12:02 - the known error keeps running, now as an expected error
```

After 10 minutes, Errors Inbox shows the new group with no occurrences in 2.2.1; with *Resolve in next version* it moves to **Resolved**.



Optional — See a regression

Switch the bug back on (`pattern on TravellersOptionBox`) for five minutes: the resolved group reopens as *Unresolved* with a **regression** tag. Switch it off again afterwards.

Check your work

- [ ] Errors carry `tags.service.version`; the workload *easyTravel* scopes Errors Inbox.
- [ ] You can show the three facts that prove 2.2.0 introduced the error.
- [ ] The new group is assigned, commented and resolved by 2.2.1; the known error is expected.
- [ ] `TravellersOptionBox` is off. Leave `JourneySearchError500` **on** — it is background noise for Lab 2.2.

Answer. 1 — Users impacted: one user retrying 50 times produces 50 occurrences but one unhappy customer. Priority follows impact on people (and business journeys), not raw counts.

Lab 2.2 — Convert a noisy static alert into a tuned baseline NRQL condition

Goal: a static p95 latency alert pages the team every time latency blips. Generate realistic noise and one real slowdown, measure the noise from `NrAiIncident`, and replace the alert with a tuned **baseline** (anomaly) condition — then compare both on the same traffic.

Time: 60 minutes (about 25 minutes of it is the noise script running). **Module:** 5.

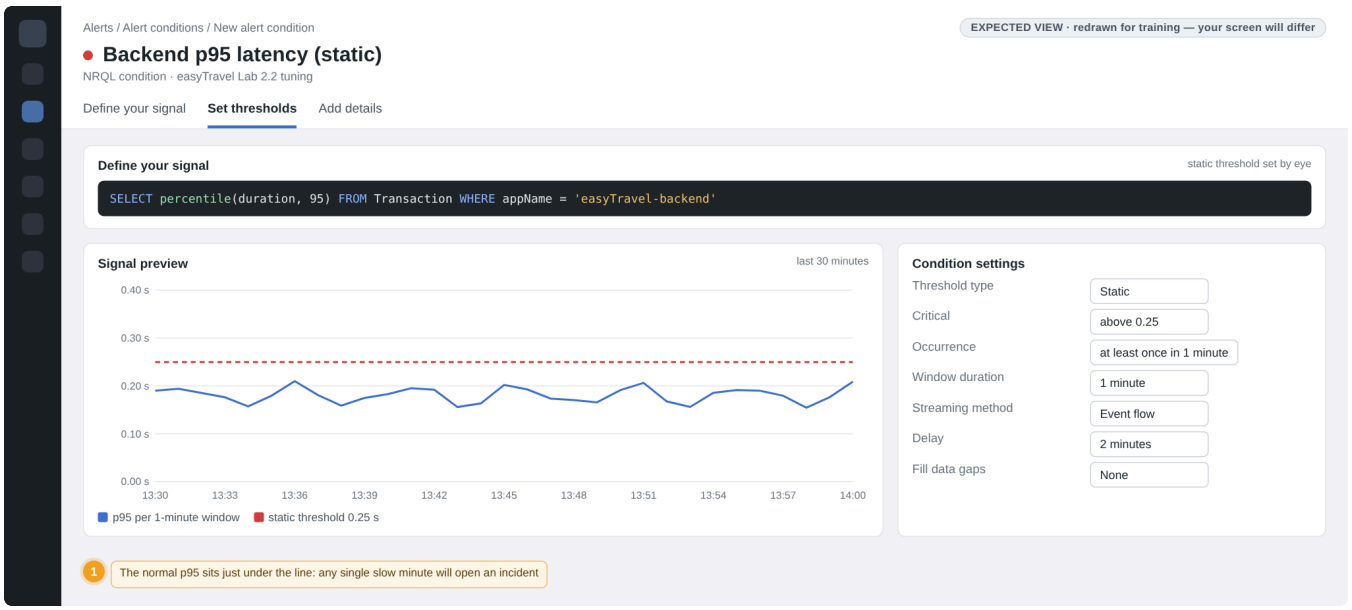
Step 1 — Find normal

```
SELECT percentile(duration, 95) AS 'p95 (s)' FROM Transaction
WHERE appName = 'easyTravel-backend' SINCE 1 hour ago TIMESERIES 1 minute
```

Note the highest normal value (for example 0.21 s). The noisy alert below uses a threshold just above it (if the blips in Step 4 do not cross it, lower it a little) — the way many real alerts are set: once, by eye, on a quiet day.

Step 2 — Create a policy and the noisy static condition

1. **Alerts → Alert policies → New alert policy.** Name: `easyTravel Lab 2.2 tuning`. Issue creation preference: **One issue per condition and signal** (so every incident is visible). Save.
2. **Alerts → Alert conditions → New alert condition → Write your own query.**
   - Query: `SELECT percentile(duration, 95) FROM Transaction WHERE appName = 'easyTravel-backend'`
   - Window duration **1 minute**, streaming method **Event flow**, delay **2 minutes**, gap filling **None**.
   - Threshold type **Static**: Critical when the query returns a value **above 0.25** (your value from Step 1, rounded up) **at least once in 1 minute**.
   - Name: `Backend p95 latency (static)`; policy: `easyTravel Lab 2.2 tuning`. Save.



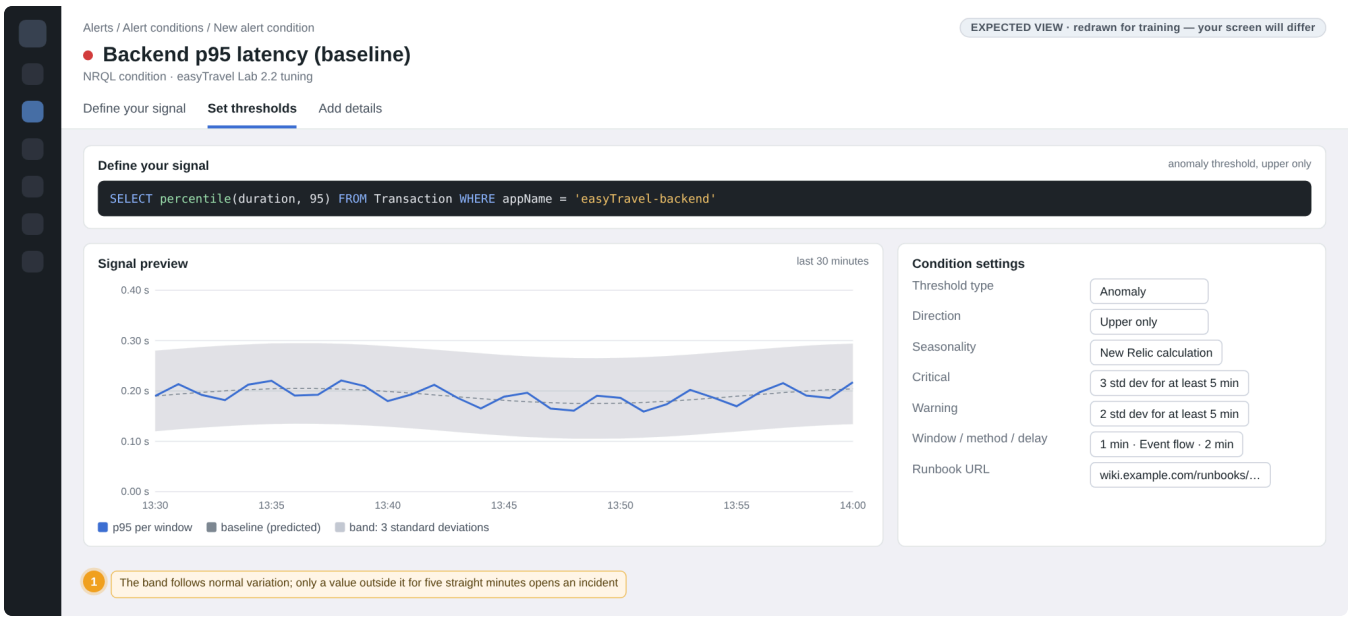
Step 3 — Create the tuned conditions (before the noise)

Create two more conditions in the same policy, with the **same query**, so all three see identical traffic:

A tuned static condition — `Backend p95 latency (static, 5 min)`: same threshold 0.25, but **for at least 5 minutes**.

A baseline condition — `Backend p95 latency (baseline)`:

- Threshold type **Anomaly**, direction **Upper only**, seasonality **New Relic calculation**.
- Critical: **3 standard deviations** above the baseline **for at least 5 minutes**. Warning: 2 standard deviations for at least 5 minutes.
- Window 1 minute, Event flow, delay 2 minutes.
- Add a runbook URL, for example `https://wiki.example.com/runbooks/easytravel-latency`.



> The baseline learns from the history it has: 1–4 weeks in a real account, about a day in this lab. The band is wider and less precise at first; that is expected.

Step 4 — Generate noise and a real problem

The script below makes four short latency blips (75 seconds each — the kind of noise that pages people for nothing) and then one sustained 10-minute slowdown (a real problem that should page someone).

```
cat > ~/nr-lab/noise.sh <<'EOF'
#!/usr/bin/env bash
# Four short DBSpamming blips (noise), then one sustained slowdown (a real
problem)
B=http://localhost:8091/services/ConfigurationService
on()  { curl -s -o /dev/null "$B/registerPlugins?pluginData=DBSpamming"
        curl -s -o /dev/null "$B/setPluginEnabled?
name=DBSpamming&enabled=true"
        echo "$(date +%H:%M:%S) DBSpamming ON  ($1)"; }
off() { curl -s -o /dev/null "$B/setPluginEnabled?
name=DBSpamming&enabled=false"
        echo "$(date +%H:%M:%S) DBSpamming OFF"; }
for i in 1 2 3 4; do on "blip $i"; sleep 75; off; sleep 150; done
on "sustained slowdown"; sleep 600; off
echo "$(date +%H:%M:%S) done"
EOF
chmod +x ~/nr-lab/noise.sh
nohup ~/nr-lab/noise.sh > ~/nr-lab/noise.log 2>&1 &
tail -f ~/nr-lab/noise.log          # Ctrl+C stops the tail, not the script
```

Lab 2.2 · Step 4 · Noise and one real problem — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ chmod +x ~/nr-lab/noise.sh
trainee@nr-lab:~/nr-lab$ nohup ~/nr-lab/noise.sh > ~/nr-lab/noise.log 2>&1 &
[1] 48211
trainee@nr-lab:~/nr-lab$ tail -f ~/nr-lab/noise.log
14:05:02 DBSpamming ON  (blip 1)
14:06:17 DBSpamming OFF
14:08:47 DBSpamming ON  (blip 2)
14:10:02 DBSpamming OFF
14:12:32 DBSpamming ON  (blip 3)
14:13:47 DBSpamming OFF
14:16:17 DBSpamming ON  (blip 4)
14:17:32 DBSpamming OFF
14:20:02 DBSpamming ON  (sustained slowdown)
14:30:02 DBSpamming OFF
14:30:02 done
```

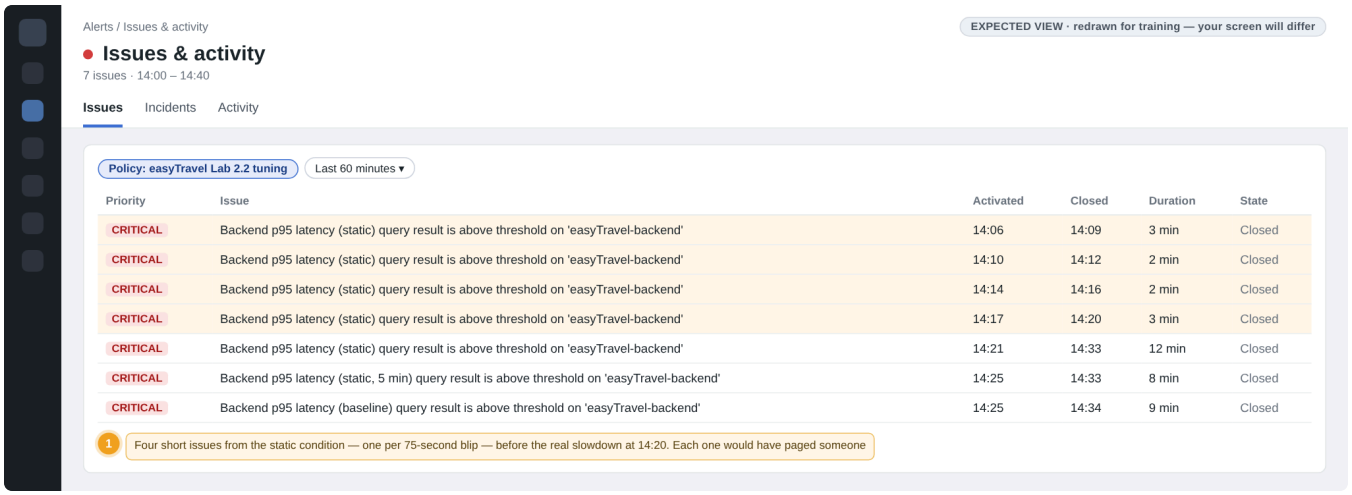
While it runs (about 25 minutes), open **Alerts → Issues & activity**, filter on the policy, and watch what each condition does.

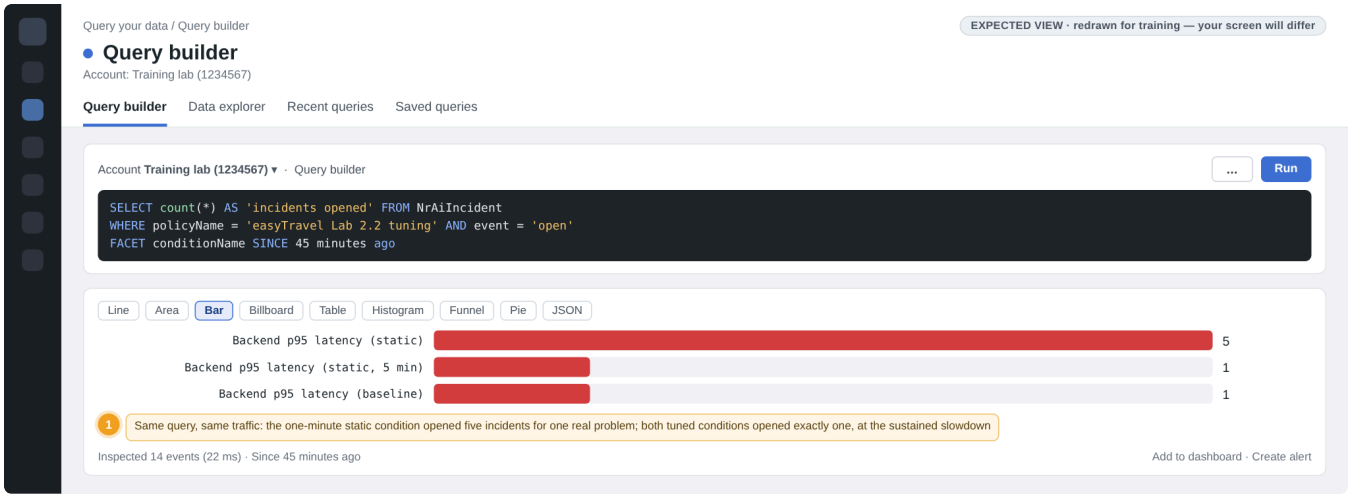
Step 5 — Measure the noise

When the script prints `done`, wait 5 more minutes and compare the three conditions on the same traffic:

```
SELECT count(*) AS 'incidents opened' FROM NrAiIncident
WHERE policyName = 'easyTravel Lab 2.2 tuning' AND event = 'open'
FACET conditionName SINCE 45 minutes ago

SELECT count(*) FROM NrAiIncident WHERE policyName = 'easyTravel Lab 2.2
tuning'
FACET conditionName, event TIMESERIES 5 minutes SINCE 45 minutes ago
```





Condition	Expected incidents	Why
Static, at least once in 1 min	~5 (each blip + the slowdown)	Any single breaching minute fires; each blip opens and closes an incident
Static, for at least 5 min	1	Blips last 75 s and never fill 5 consecutive windows

Condition	Expected incidents	Why
Baseline, 3 σ for at least 5 min	1 (possibly a warning earlier)	Same duration logic, and the band follows normal variation instead of a fixed guess

Question 1. Both tuned conditions caught only the real problem. Why keep the baseline one rather than the tuned static one?

Step 6 — Review priority, duration and threshold

Finish the conversion:

1. **Disable** `Backend p95 latency (static)` — it paged five times for one real problem.
2. Demote `Backend p95 latency (static, 5 min)` to **Warning** priority, or delete it if the baseline condition covers the need.
3. Keep `Backend p95 latency (baseline)` as **Critical**, with the runbook URL.
4. Write down the review: date, incidents before/after, what changed. This is the record that stops someone "fixing" it back.

Optional — The same condition as code

Get the policy ID (*Alerts* → *Alert policies* → the policy; the ID is shown on the page and in its URL), then create the baseline condition through NerdGraph — the way conditions are kept under version control:

```
set -a; source ~/nr-lab/.env; set +a
POLICY_ID=<policy id>
cat > /tmp/baseline.graphql <<EOF
mutation {
  alertsNrqlConditionBaselineCreate(accountId: $NEW_RELIC_ACCOUNT_ID,
policyId: "$POLICY_ID", condition: {
    name: "Backend p95 latency (baseline, as code)"
    enabled: true
    nrql: { query: "SELECT percentile(duration, 95) FROM Transaction WHERE
appName = 'easyTravel-backend'" }
    baselineDirection: UPPER_ONLY
    signalSeasonality: NEW_RELIC_CALCULATION
    terms: [{ priority: CRITICAL, operator: ABOVE, threshold: 3,
thresholdDuration: 300, thresholdOccurrences: ALL }]
    signal: { aggregationWindow: 60, aggregationMethod: EVENT_FLOW,
aggregationDelay: 120 }
    violationTimeLimitSeconds: 86400
  }) { id name }
}
EOF
jq -n --rawfile q /tmp/baseline.graphql '{query: $q}' \
  | curl -s https://api.newrelic.com/graphql -H 'Content-Type:
application/json' -H "API-Key: $NEW_RELIC_API_KEY" -d @- | jq .
```

Lab 2.2 · Optional · Baseline condition as code — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ jq -n --rawfile q /tmp/baseline.graphql '{query: $q}' | curl -s
                         https://api.newrelic.com/graphql -H 'Content-Type:
                         application/json' -H "API-Key: $NEW_RELIC_API_KEY" -d @- | jq .
{
  "data": {
    "alertsNrqlConditionBaselineCreate": {
      "id": "51234567",
      "name": "Backend p95 latency (baseline, as code)"
    }
  }
}
the condition appears in the policy; keep /tmp/baseline.graphql in Git next to your other
    conditions
```

Check your work

- [ ] You can show, from `NrAiIncident`, how many incidents each condition opened for the same traffic.

- [ ] The noisy static condition is disabled; the baseline condition is critical with a runbook URL.
- [ ] `noise.sh` has finished and DBSpamming is **off** (`pattern off DBSpamming`).

Answer. 1 — The static threshold is still a fixed guess: if normal latency rises (more traffic, a heavier page) it starts firing all day, and if normal falls, real slowdowns stay under it. The baseline moves with normal and only needs its sensitivity reviewed.

Lab 2.3 — Run a simulated P1 incident from alert to acknowledgement to root cause

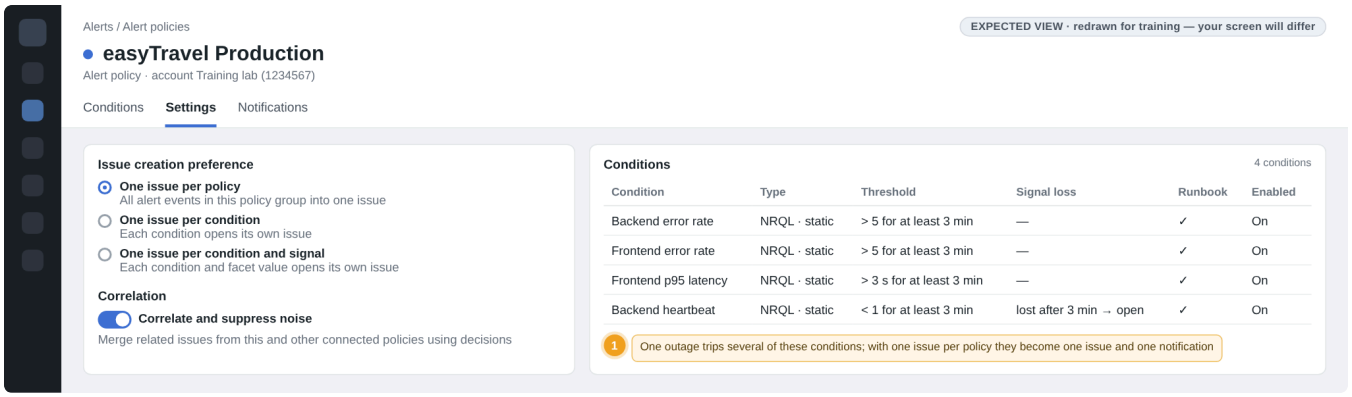
Goal: set up alerting the way a production team would — one policy, an email destination, a P1 workflow, a maintenance-window muting rule — then cause an outage and run it as a P1: notification, acknowledgement, impact, location, root cause, recovery and timeline.

Time: 60 minutes. **Module:** 6.

Step 1 — The production policy and its conditions

Alerts → Alert policies → New alert policy:

- Name `easyTravel Production`; issue creation preference **One issue per policy**; turn on **Correlate and suppress noise**.



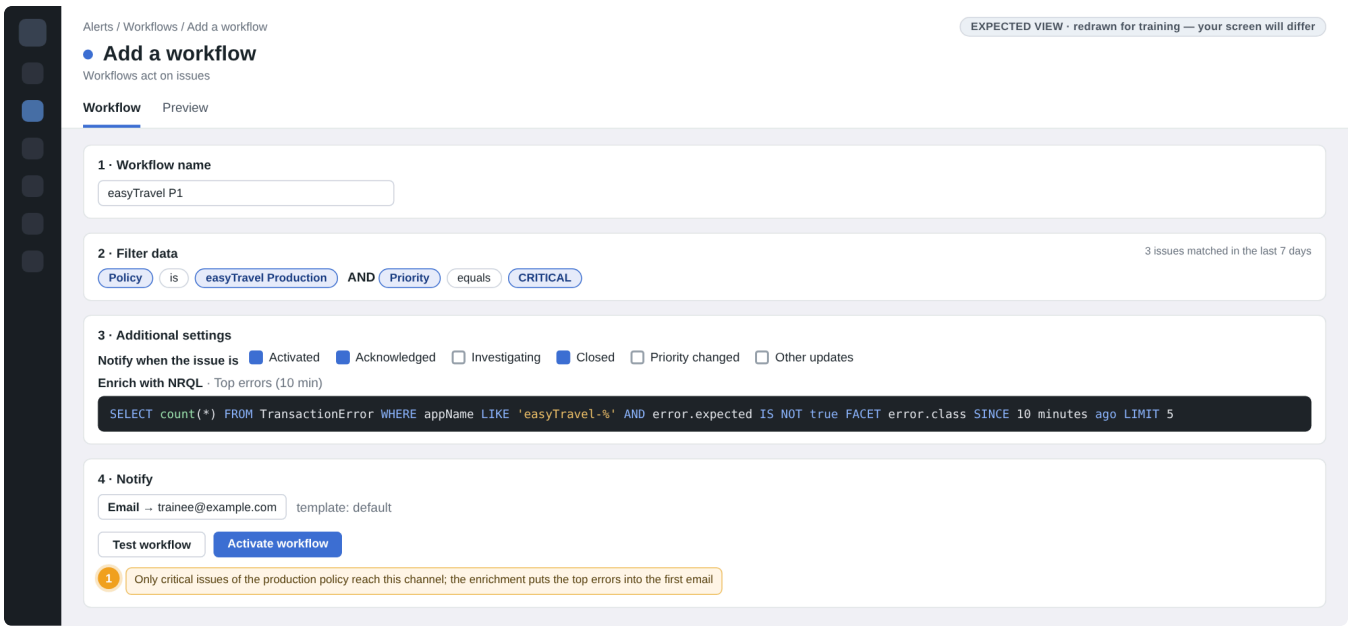
Add four NRQL conditions (window 1 minute, Event flow, delay 2 minutes unless noted). Paste the runbook URL `https://wiki.example.com/runbooks/easytravel-p1` into each.

Name	Query	Threshold
`Backend error rate`	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction WHERE appName = 'easyTravel-backend'`	Static, Critical above 5 for at least 3 min
`Frontend error rate`	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction WHERE appName = 'easyTravel-frontend'`	Static, Critical above 5 for at least 3 min
`Frontend p95 latency`	`SELECT percentile(duration, 95) FROM Transaction WHERE appName = 'easyTravel-frontend'`	Static, Critical above 3 for at least 3 min
`Backend heartbeat`	`SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-backend'`	Static, Critical below 1 for at least 3 min; **signal loss**: *signal is lost after* 3 minutes → *open new incident*

Because the known journey-search error is now expected (Lab 2.1), it does not count towards the error rates.

Step 2 — Destination and P1 workflow

1. **Alerts → Destinations → Email:** add your email address and save. (Or create it while adding the workflow.)
2. **Alerts → Workflows → Add a workflow:**
   - Name `easyTravel P1`.
   - **Filter:** policy is `easyTravel Production` **and** priority equals `CRITICAL`.
   - **Notify when:** *Activated*, *Acknowledged*, *Closed*.
   - **Enrichment** (NRQL), named `Top errors (10 min)`: `SELECT count(*) FROM TransactionError WHERE appName LIKE 'easyTravel-%' AND error.expected IS NOT true FACET error.class SINCE 10 minutes ago LIMIT 5`
   - **Channel:** Email → your destination.
   - **Test** the workflow, then **Activate** it.



Step 3 — Planned maintenance under a muting rule

Operations will take the backend down for four minutes. Nobody should be paged.

1. **Alerts → Muting rules → Add a rule:** name `easyTravel backend maintenance`; condition `policyName` **equals** `easyTravel Production`; schedule **once**, from now for 20 minutes, in your time zone; when the window ends: **Notify**. Save — the status shows *Active*.

> Image — Expected view — the muting rule: one policy, a fixed 20-minute window, notify w…

2. Do the maintenance:

```
date +%H:%M; etdc stop backend
sleep 240
etdc start backend && sleep 90 && date +%H:%M
```

> Image — Expected output — the backend stopped for four minutes of planned maintenance, …

3. In **Issues & activity**, an issue appears for the policy (heartbeat signal loss and frontend errors) marked **Muted**; **no email** arrives.
4. When the frontend has recovered (about 5 minutes), **disable** the muting rule so it cannot hide the next step.

Question 1. Why is a muting rule with a fixed end time and one policy safer than one matching `IS_ANY` with no end?

Step 4 — The P1: cause the outage

Check that the policy has no open issue, then:

```
date +%H:%M
etdc stop mongodb
sleep 60; curl -s -o /dev/null -w "home page: HTTP %{http_code} in %
{time_total}s\n" http://localhost/
```

Image — Expected output — MongoDB stopped at the start of the P1; the home page fails a…

Within about five minutes, the backend error rate, frontend error rate and frontend p95 conditions breach. All three incidents join **one issue** (one issue per policy), and the workflow sends **one email** with the *Top errors* enrichment.

Step 5 — Acknowledge

Open the issue from the email link (or **Issues & activity**) and click **Acknowledge**. Note the time from the first notification to acknowledgement — this is your *time to acknowledge*. The workflow sends an *Acknowledged* notification.

Image — Expected view — the P1 issue: three correlated incidents, impacted entities, ac…

Step 6 — Investigate with the runbook

Work through the runbook from the study guide and write each answer in your incident notes:

1. **Impact.** Frontend error rate and the booking journey: are customers affected, how many? (Errors Inbox *users impacted*; `SELECT uniqueCount(session) FROM PageView SINCE 10 minutes ago` if browser monitoring is enabled.)
2. **Locate.** Service map of easyTravel-backend; **Distributed tracing** filtered on errors → the deepest error span is a MongoDB datastore call with `MongoTimeoutException` (as in Lab 1.2).

3. **Check for change.** No deployment marker since 2.2.1, which has run cleanly for an hour. In Errors Inbox the MongoDB timeout group from Day 1 is back (as a regression if you resolved it) from the start of the outage, in the same version as before. Not a release: an infrastructure failure.
4. **Root cause.** The backend cannot reach its database: MongoDB is down.

```
SELECT count(*) FROM Span WHERE error.class IS NOT NULL
FACET appName, name, error.class SINCE 10 minutes ago
```

Step 7 — Mitigate and close

```
etdc start mongodb
sleep 120; curl -s -o /dev/null -w "home page: HTTP %{http_code} in %
{time_total}s\n" http://localhost/
```

Image — Expected output — MongoDB restored; the home page answers normally again

The conditions recover and their incidents close; when the last one closes, the issue closes and the workflow sends a *Closed* email.

Step 8 — The timeline

```
SELECT timestamp, event, priority, conditionName, title, muted, closeCause
FROM NrAiIncident
WHERE policyName = 'easyTravel Production' SINCE 90 minutes ago ORDER BY
timestamp ASC LIMIT MAX
```

Image — Expected view — the post-incident timeline from NrAiIncident: the muted mainten…

Write a five-line summary in your notes: **start** (first incident), **acknowledged** (time), **impact** (journeys and users), **root cause** (MongoDB unavailable), **recovery** (time) — and one action: *add a MongoDB availability condition to the infrastructure policy so the cause pages directly next time.*

Step 9 — Clean up

- Disable the `easyTravel P1` workflow (or keep it and set the conditions to disabled) so lab exercises do not email you later.
- Delete the muting rule.
- Turn off the remaining pattern: `pattern off JourneySearchError500`.

Check your work

- [ ] One issue held all three P1 incidents, and one email (plus *Acknowledged* and *Closed*) was sent.
- [ ] The maintenance produced a **muted** issue and no email.
- [ ] You can state the root cause with evidence (deepest error span, timeline) and the time to acknowledge.
- [ ] MongoDB is running, all patterns are off, the workflow is disabled and the muting rule deleted.

Answer. 1 — A narrow rule mutes only what is under maintenance, and a fixed end means a forgotten rule expires by itself. `IS_ANY` with no end mutes every alert in the account until someone notices — exactly the outage you would want to hear about.

Appendix — Reset, troubleshooting and end-of-day checklist

Reset everything

```
for p in DBSpamming DBSpammingAuthWithAppDeployment JourneySearchError500
TravellersOptionBox; do pattern off $p; done
etdc start mongodb backend frontend
pkill -f noise.sh || true
etdc ps --format "table {{.Name}}\t{{.Status}}"
```

In New Relic: disable or delete the Lab 2.2 and Lab 2.3 policies, the `easyTravel P1` workflow and the muting rule if the account is shared.

Troubleshooting

Symptom	Check	Fix
No version in Errors Inbox / `tags.service.version` empty	`etdc exec backend sh -c 'echo $NEW_RELIC_METADATA_SERVICE_VERSION'`	Recreate with `etdc up -d frontend backend` after editing `.env`; `restart` does not apply new variables
No new error group after `TravellersOptionBox`	`SELECT count(*) FROM TransactionError FACET appName, error.class SINCE 15 minutes ago`	The pattern may have been reset by a restart — run `pattern on TravellersOptionBox` again
Expected error not applied	`grep -A3 error_collector ~/nr-lab/agent/*/newrelic.yml`	Indentation: `error_collector` must be two spaces in, under `common:`; class and message phrase must match exactly
Static conditions never fire	Condition preview chart	Threshold above the values you generated, or the query returns no data (check app name)
Baseline band very wide	Condition preview	Normal with little history; it narrows over days
No email from the workflow	*Workflows* → the workflow → *Test*; spam folder	Filter does not match (policy name, priority); destination not verified
P1 creates several issues	Policy settings	Issue creation preference must be **One issue per policy**
Muted issue still emailed	Muting rule status	Rule not *Active* yet, or condition attribute/value mismatch (matching is case-sensitive)

End-of-day checklist

- [ ] Workload *easyTravel* exists; errors carry a version.

- [ ] Release 2.2.0 is identified as the source of the traveller-options error; the group is resolved by 2.2.1.
- [ ] Noisy static condition disabled; baseline condition kept; the review is written down.
- [ ] The P1 timeline and summary are saved with your notes.
- [ ] Patterns off, MongoDB running, workflow disabled, muting rule deleted.

Sources

- New Relic docs — Errors inbox, Version tracking, Java agent error collector settings
- New Relic docs — NRQL alert conditions, Anomaly detection, Workflows, Muting rules
- Dynatrace — easyTravel problem patterns